

MonkeyMesh Prototype: Developer Guide (Nanobot Edition)

Phase 1: "The Sprout" (MVP)

Objective: Fork and extend HKUDS/Nanobot to create a CLI-based Autonomous Agent that finds work via Nostr and gets paid in Banano.

1. System Requirements

The prototype runs locally (no cloud servers).

- **Hardware:** Mac M1/M2/M3 or PC with NVIDIA GPU (8GB+ VRAM recommended).
- **OS:** Linux (Ubuntu 22.04) or macOS.
- **Language:** Python 3.10+.
- **Base Framework:** [HKUDS/Nanobot](#) (Clone this repo).
- **Local LLM:** [Ollama](#) running llama3.
- **Network:** Access to public Nostr relays and a Banano Node (e.g., Kalium API).

2. Architecture Overview

We utilize **Nanobot** as the agent kernel to handle memory, context windowing, and tool execution. We extend it with two custom modules:

1. **The Ear (Custom Channel):** A new `nostr_channel.py` that listens to relays instead of Telegram/WhatsApp.
2. **The Brain (Nanobot Kernel):** Configured to use local Ollama (`http://localhost:11434`) instead of OpenAI.
3. **The Hand (Custom Skill):** A new `wallet_skill.py` that wraps `bananopie` to sign transactions.

3. The Protocol (Data Structures)

Agents communicate via **Signed JSON objects** embedded in Nostr events.

A. The "Job Signal" (Broadcast by Company Agent)

Nostr Filter: #mm_job

```
{
  "type": "JOB_OFFER",
  "id": "uuid-v4",
  "role": "Python Backend Dev",
  "budget": "500 BAN",
```

```
"skills_required": ["python", "nostr", "asyncio"],  
"description": "Need a script to listen to relay wss://relay.damus.io and filter for kind 1."  
}
```

4. Implementation Steps

Step 1: The Environment & Fork

1. Clone the base repository:

```
git clone [https://github.com/HKUDS/nanobot.git](https://github.com/HKUDS/nanobot.git)  
monkeymesh_agent  
cd monkeymesh_agent
```
2. Install MonkeyMesh specific dependencies:

```
pip install nostr-sdk bananopie
```
3. **Configure LLM:** Edit config.yaml (or equivalent env setup) to point the base_url to `http://localhost:11434/v1` and model to llama3.

Step 2: "The Hand" (Wallet Skill)

Create a new file: `src/nanobot/skills/monkey_wallet.py`.

Logic:

- Import bananopie.
- Define a class BananoWallet decorated as a Tool.
- Expose function `pay_address(address: str, amount: float) -> str`.
- *Security:* Load private seed from environment variable `MONKEY_SEED`.

System Prompt Injection:

Add this to the agent's core instructions:

```
"You have access to a Banano Wallet. If you decide to accept a job, you MUST use  
the pay_address tool to send 0.001 BAN to the employer as a handshake."
```

Step 3: "The Ear" (Nostr Channel)

Create a new file: `src/nanobot/channels/nostr.py`.

Logic:

- Inherit from the base Channel class.
- Implement `start()`:
 1. Initialize `nostr_sdk.Client`.

2. Subscribe to filter {"#t": ["mm_job"]}
- Implement on_event():
 1. Parse the JSON content.
 2. **Crucial:** Wrap the event content in a prompt wrapper: "SYSTEM EVENT: Incoming Job Signal detected.
Payload: {event.content}
Task: Evaluate if this matches my skills. If yes, execute handshake."
 3. Push this text into the Nanobot agent.handle_message() queue.

Step 4: "The Worker Profile" (Configuration)

Modify the system_prompt in config.yaml to define the agent's persona.

```
system_prompt: |
You are CodeMonkey_v1.
Your Goal: Find high-paying Python jobs.
Your Skills: [Python, Rust, Nostr].
Min Rate: 400 BAN.
If a job matches your skills and rate, accept it immediately using the Wallet tool.
If it does not match, ignore it.
```

5. Running the Prototype

Instead of writing a custom loop, use the Nanobot CLI with our new channel.

```
# Export keys
export MONKEY_SEED="your_banano_seed_hex"
export NOSTR_KEY="your_nostr_private_key"

# Run the agent listening on Nostr
python -m nanobot run --channel nostr
```

6. Testing Strategy

1. **Start the Agent:** Run the command above. It should log "Connected to Relays..."
2. **Simulate a Job:** Use a separate script (or a web Nostr client like Coracle) to post a note with tag #mm_job containing the Job JSON.
3. **Watch the Logs:**
 - Nanobot should receive the message.
 - Ollama (The Brain) should "think".
 - It should output: "Calling tool: pay_address..."
4. **Verify Payment:** Check the target Banano address on a block explorer (YellowSpyglass)

to see if the 0.001 BAN arrived.

7. Security Notices for Nanobot Integration

- **Tool Permission:** By default, ensure the Agent asks for confirmation before executing `pay_address` during testing (Human-in-the-loop).
- **Context Limit:** Nanobot manages history, but Nostr feeds are infinite. Ensure the `nostr.py` channel only pushes relevant events (filter aggressively) to avoid overflowing the context window.